

Exploring the Ability of a Linear Regression Model in Predicting Housing Prices

Mathematics: Analysis and Approaches HL

Internal Assessment

Nikita Manushin

June 2026

1 Introduction

1.1 Personal Connection

When I started learning math, in the very first grade, I have always wondered how connected, the numbers and the real world are. In the past few years Neural Networks became so powerful they could communicate like humans, all of that using tons of numbers, that create words and sentences. This linear regression model I am about to present, is an example, how what seem to be meaningless numbers can make predictions about something that humans care about.

1.2 Aims of the Investigation

In this paper, I will explore how a linear regression model can be trained using gradient descent to predict prices of apartments in Amsterdam on the moment of June 2026. The features of apartments will act as the input data, I will be

using living area, construction year, and distance from the center of Amsterdam as my independent variables. Gradient descent uses loss functions to figure out the direction and magnitude of how the model must be adjusted to predict the output more accurately. I will be comparing three loss functions: MSE (mean squared error), MAE (mean absolute error) and Huber's, to analyse and experimentally suggest which method would be the most efficient for my case.

There is another way of finding parameters for a linear-regression model that is called ordinary least squares (OLS), and the reason I am using gradient descent is that in this paper I want to resemble the process of machine learning and a neural network, which is what I took my inspiration from for this model. Furthermore, this allows me to observe how model parameters are optimized iteratively using the gradients of loss functions.

1.3 Research Question

To what extent different loss functions (MSE, MAE and Huber) affect the accuracy of linear-regression models that are trained using gradient descent to predict current prices of apartments in Amsterdam.

2 Theory

2.1 A Simple Linear Regression Model

Linear regression is a tool used to predict a continuous dependent output variable based on one or more independent input variables by fitting a line, a surface or a higher-dimensional shape to minimize the loss.

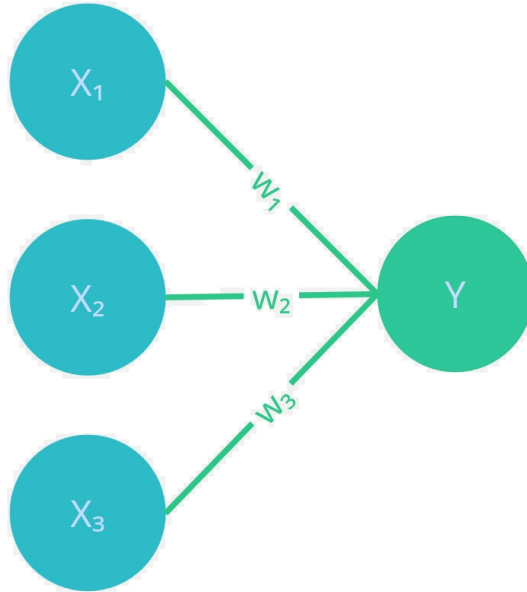


Figure 1: Model Representation

This is the structure of the linear regression model I will be using. It consists of independent variables x_1 , x_2 , x_3 called inputs and a dependent variable y called output. Inputs describe features of a particular apartment: x_1 represents the living area, x_2 is the construction year and x_3 represents the distance between the apartment and the center of Amsterdam. The output y is the price of the apartment.

A linear function that has three inputs should have three slopes, which in machine learning are called weights w_n , where w_n is the weight of x_n . Weights are constants that are used to create the linear function. However, a linear function also requires a y-intercept, which in machine learning is called a bias b . The whole function of y looks like:

$$y = w_1x_1 + w_2x_2 + w_3x_3 + b$$

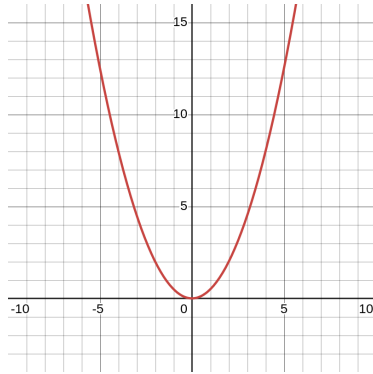
The function suggests that the price of an apartment can be calculated using those three inputs and by multiplying them by their weights and adding a bias. The weights and biases are constants.

2.2 Machine Learning

The goal of machine learning is to find the specific weights and bias that will predict the output the most accurate. In order to quantify the accuracy of a particular set of weights and biases a loss function is used. A loss function L has two independent variables: y and t , y is the value predicted by the model using those weights and biases and t is the target value. There are lots of different loss functions, but I will only be comparing three: MSE, MAE and Huber.

2.3 Loss Functions

2.3.1 MSE



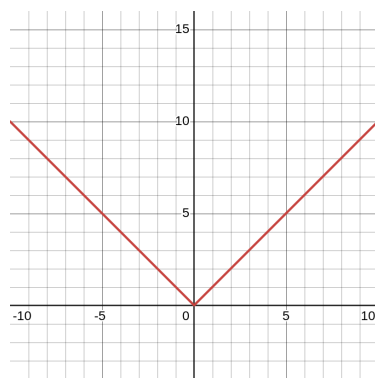
$$L_{\text{MSE}} = \frac{1}{2}(y - t)^2 \quad (1)$$

$$L'_{\text{MSE}} = y - t$$

Figure 2: MSE loss

MSE uses squaring to create a positive difference between y and t : $(+5)^2 = (-5)^2$. The half $\frac{1}{2}$ is there just to make the derivative look cleaner: $y - t$. However, the squaring penalizes large errors disproportionately. This could be good, as it gives more importance to cases when the model performs the poorest, however squaring makes the learning sensitive to outliers.

2.3.2 MAE



$$L_{\text{MAE}} = |y - t| \quad (2)$$

$$L'_{\text{MAE}} = \frac{|y - t|}{y - t}$$

Figure 3: MAE loss

MAE uses absolute value to find the error. This makes all of the errors proportionately important. This makes it less sensitive to outliers. However, in datasets where large errors are significant it would perform poorly. Furthermore, this loss function is not differentiable at $y = t$, as in $\frac{|0|}{0}$ there is division by 0.

Another proof that this function is not differentiable at 0 is using limits from both sides.

let $h = y - t$

$$\lim_{h \rightarrow 0^+} \frac{|h|}{h} = \lim_{h \rightarrow 0^+} \frac{+h}{+h} = +1$$

$$\lim_{h \rightarrow 0^-} \frac{|h|}{h} = \lim_{h \rightarrow 0^-} \frac{+h}{-h} = -1$$

$$+1 \neq -1$$

$$\lim_{h \rightarrow 0^+} L' \neq \lim_{h \rightarrow 0^-} L'$$

$\therefore \lim_{h \rightarrow 0}$ does not exist

For my calculations I will define the gradient of MAE at $y = t$ as 1, to make the function continuous.

2.3.3 Huber

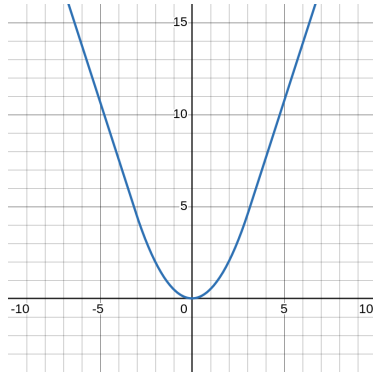


Figure 4: Huber loss

$$\text{let } h = y - t \tag{3}$$

$$L_{\text{Huber}} = \begin{cases} \frac{1}{2}(h)^2, & |h| \leq \delta \\ \delta|h| - \frac{1}{2}\delta^2, & |h| > \delta \end{cases}$$

$$L'_{\text{Huber}} = \begin{cases} h, & |h| \leq \delta \\ \delta \frac{|h|}{h}, & |h| > \delta \end{cases}$$

Huber takes advantages of both MSE and MAE, by combining two functions into one piecewise, that connects at $x = \delta$. For outliers Huber uses MAE, which makes their impact less significant. On the other hand, MSE makes the other data points have different importances depending on the errors' magnitude. The

only problem is finding the most efficient δ . There isn't a straight forward way of finding the constant. Usually values like $\delta = 0.5, 1.0, 1.345, 2.0, 5.0$ are used. In order to find the best coefficient for my data I have tested the values above.

2.4 Gradient Descent

2.4.1 Visualize

Gradient descent is the what calculates a particular weight or bias during the machine learning. As shown on the diagram below, we could plot the loss function, in this case MSE is used, and drop the weight on a random point. Now, after calculating the gradient of the loss function we could change the weight in the correct direction to get closer to the local minimum. On this example only one weight varies, and every other weight and bias are fixed. In the actual machine learning, which will be done further, every weight and bias will have its own curve, in which they all will be adjusted to approach the minimum of the function simultaneously.

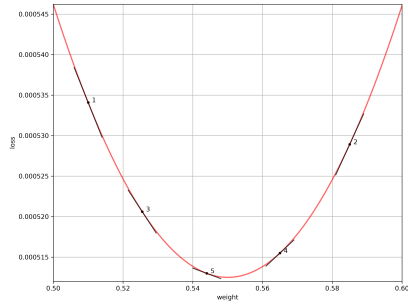


Figure 5: Gradient Descent

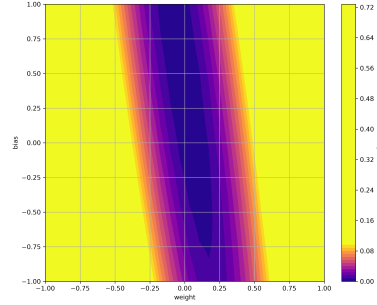


Figure 6: Contour Map

To visualize how gradient descent would look while multiple variables are changing, I have computed this contour map where y, t are fixed and w_2 and w_3 are random constants, and depending on how w_1 and b are tweaked (by the

axis) the loss is changing (indicated with the color). (I have picked only two variables because it would be almost impossible to visualize a five dimensional function like L_{MSE} even with y and t fixed).

We can see the purple spots on the diagram which represent the minimums or the lower loss areas. The gradient descent should, if set up properly, tweak w_1 and b that way that the loss will move to those areas.

2.4.2 Differentiate to find the Gradients

The gradient of weight in this graph is just the partial derivative of the loss by the weight. I will start of with writing down all of the equations we have:

$$L(y, t) = \frac{1}{2}(y - t)^2$$
$$y = w_1x_1 + w_2x_2 + w_3x_3 + b$$

In order to derive further I will have to use partial derivatives. A partial derivative is a type of derivative that applies to multi dimensional/variable functions when we have to figure out the rate of change of this function in one particular dimension. To partially derive a function I have to treat all of the other variables as constants just for the derivative. First I derive MSE loss with respect to y , so treat t as a constant:

$$\frac{\partial L}{\partial y} = y - t \tag{4}$$

Now if I want to partially derive y with respects to w_1 , to see how the answer changes with the first weight (this is the one dimension):

$$\begin{aligned}\frac{\partial y}{\partial w_1} &= \frac{\partial y}{\partial w_1}(w_1 x_1) + \frac{\partial y}{\partial w_1}(w_2 x_2) + \frac{\partial y}{\partial w_1}(w_3 x_3) = \\ &= x_1 + 0 + 0 = x_1\end{aligned}$$

Now using the chain rule $\frac{du}{dv} = \frac{du}{dn} \times \frac{dn}{dv}$, I can combine two derivatives and find the gradient of w_1 with respect to L :

$$\begin{aligned}\frac{\partial L}{\partial w_1} &= \frac{\partial L}{\partial y} \times \frac{\partial y}{\partial w_1} = \\ &= (y - t) \times x_1\end{aligned}\tag{5}$$

This needs to be done with every other weight and the bias:

$$\begin{aligned}\frac{\partial L}{\partial w_2} &= \frac{\partial L}{\partial y} \times \frac{\partial y}{\partial w_2} = (y - t) \times x_2 \\ \frac{\partial L}{\partial w_3} &= \frac{\partial L}{\partial y} \times \frac{\partial y}{\partial w_3} = (y - t) \times x_3 \\ \frac{\partial L}{\partial b} &= \frac{\partial L}{\partial y} \times \frac{\partial y}{\partial b} = (y - t)\end{aligned}\tag{6}$$

2.4.3 Tweak the Weights and Biases

This is how the model knows in what direction to change the weights/bias and how important it is to change a weight/bias to reduce the loss. To change a weight/bias we could just subtract the derivative from the weight:

$$\begin{aligned}\text{let } h &= \text{weight } w_n \text{ or bias } b \\ h &= h - \frac{\partial L}{\partial h}\end{aligned}$$

However, the derivative might be too big, and the weight/bias could change

too much. This could cause instability, as the weight could overshoot all the time, and it might never reach the minimum. To address this a learning rate constant is introduced - η . The derivative is multiplied by it to ensure that the steps are small so that the minimum is achieved no matter what. The formula for weight change becomes:

$$h = h - \frac{\partial L}{\partial h} \times \eta \quad (7)$$

η is usually somewhere around 0.1 to 0.001. This is done to every weight/bias during the machine learning process. So for example, this would be the formula for the first weight changing with learning rate η of 0.1, using MSE loss:

$$w_1 = w_1 - \frac{\partial L}{\partial w_1} \times 0.1$$

$$w_1 = w_1 - (y - t) \times x_1 \times 0.1$$

One of the things I have not mentioned yet is where I got the x and t from. As I said x is the living area, and t is the price of an apartment. I have collected data from funda.nl and I only picked the apartments that are in Amsterdam and that are for sale in June of 2026. In total, I have collected around 4500 apartments with their respective prices and features.

Because the function does calculation of only one apartment at a time, I took average of all losses for each apartment with a certain weight and bias and that is what is being presented on the previous graphs. That is why I have not used x nor t as arguments for functions y and L , for the process of learning it is treated as a constant. The weight and bias are the independent variables that must be changed. After the model has finished its learning and, therefore, has constant weight and bias, the function will become $y(x) = wx + b$ and x will be the independent variable again.

2.5 Normalization

Another thing that must be done for a model to work is normalization of inputs and outputs. Normalization is the process of fitting a set of values into the range between 0 to 1.

$$x_{\text{norm}} = \frac{x_{\text{raw}} - x_{\text{min}}}{x_{\text{max}} - x_{\text{min}}} \quad (8)$$

Where x_{raw} is the value to be normalized; x_{min} and x_{max} are the minimum and maximum values of the same type in the dataset; x_{norm} is the normalized value in the range 0 - 1.

This is done for every input: price, living area and distance, and the output: the price of the apartment. Normalization will ensure that weights are also in a small range, usually from -1 to 1, rarely might reach -2, or 2.

For my data the min and max for prices and living areas are respectively:

$$x_1 = 19 - 500 \text{ m}^2$$

$$x_2 = 1507 - 2028 \text{ year}$$

$$x_3 = 0.07 - 9.98 \text{ km}$$

$$y = 189 - 9850 \text{ k€}$$

Also, I divide the price by 10^3 , keeping it in thousand of euros (k€). This makes the numbers clearer and potentially has a small impact on the training, as the data is shrank by 1000 and the difference between prices is lower.

All of the data must be normalized before it is used in the training, this is how the function for x_1 looks like:

$$x_1 = \frac{x_{\text{raw}} - 19}{500 - 19}$$

2.6 Training Process

I have a few thousand apartments data from which my model must learn. The term epoch is introduced at this point, which refers to a single, complete pass of the entire training dataset through the model. I will usually be doing 30-100 epochs for this model. The dataset includes values: (x_1, x_2, x_3, t) for each apartment. I have created a program that will do the training so I don't have to do it manually.

As I mentioned earlier, the dataset includes around 4500 entries; it is exactly 4577 apartments. The data must be separated into training and test datasets. The training entries are used to train the model, in the process mentioned above. On the other hand, the test dataset is used to test the accuracy of predictions of the model. The test process illustrates how good the model performs on apartments it has never seen before. To separate the dataset into training and test datasets, I will be using a 0.95 ratio - 4348 and 229 entries, respectively.

Here is step by step overview of how my program works:

- Prepare datasets

Find min and max for each variable x_1, x_2, x_3, t and normalize the them. Separate the dataset randomly into two parts: test and training. The ratio is 0.95, therefore there are 229/4348 data entries.

- Initialize the weights and bias

$$w_n \sim \frac{1}{3}U(0,1)$$
$$b = 0$$

$U(0,1)$ - uniform distribution from 0 to 1. $\frac{1}{3}$ because there are 3 inputs, making the initial prediction smaller.

- Training

One epoch:

I extract (x_1, x_2, x_3, t) from a entry in the dataset, with those I calculate gradients for the weights and the bias using equations 5 and 6. Using those gradients new weights and bias are calculated using equations 7. Then I extract the next entry from the dataset, and repeat the process until I do that for the whole training dataset.

This is done for a certain number of epochs. As a result I get certain values for w_1, w_2, w_3, b . The model is ready to predict house prices.

- Testing

For each of the entries from the testing dataset extract (x_1, x_2, x_3, t) , using them I calculate y , using $w_1x_1 + w_2x_2 + w_3x_3 + b$. To compare predicted y and the actual t I use two methods: normalized losses such as MSE, MAE or Huber depending on what I have used during the training; and non normalized mean average error:

$$\frac{1}{n} \sum_{i=1}^n |y_i - t_i|$$

Where n is the number of entries in the testing dataset.

3 Experiment

3.1 Dataset Problem

As I have mentioned earlier, I split the total dataset into two: training and testing. However, this approach might result in incorrect comparison of different losses as training set might have more outliers in some cases which would affect

MSE loss differently than MAE loss. This forced me to instead of training one model, train several, each shuffled differently and then compare the mean results with other models. Furthermore the shuffling random seeds are synchronized across losses. This improves analytical accuracy of the results I will get.

The mean absolute value across multiple models looks like:

$$\frac{1}{nm} \sum_{i=1}^n \sum_{j=1}^m |y_{ij} - t_{ij}|$$

Where n is the number of differently trained models, and m is the number of entries in training dataset.

3.2 Goal

My goal is to compare linear-regression models that use MSE, MAE and Huber losses on how well they perform at predicting housing prices. The quantity that I will be comparing is the non-normalized mean absolute error - the average difference between the predicted and actual prices in k€.

3.3 Huber Constant

As I have mentioned earlier Huber loss requires the constant δ , however it is difficult to analytically find it. Therefore I have wrote a script that trains this model using huber loss with different δ . I have trained 50 models for each huber constant with different shuffles and with the training of 50 epochs. This is the results I got:

δ	0.1	0.175	0.25	0.325	0.4
$L_{\text{Huber}} (10^{-4})$	4.9901	4.9066	4.8946	4.8942	4.8961
$MAE_{\text{norm}} (\text{k€})$	138.68	144.58	147.46	149.56	150.53

3.4 Results

Finally everything is set up. I have trained 100 models with each loss function for 100 epochs. These are the results:

	loss	MAE_{norm}
MSE	0.00050421	152.32
MAE	0.00076356	189.12
Huber	0.00061698	144.87

MSE had the least loss, and Huber performed with the least MAE_{norm} .

4 Evaluation

4.1 Achieving the Aim

The aim for this paper was to figure out whether a linear regression model could accurately predict prices of apartments in Amsterdam and compare performance of different loss functions on the mean absolute error after testing. The experiment resulted with the least MAE_{norm} being achieved by the Huber loss. The model can predict apartment prices with the error of 144.87 k€.

4.2 Data

I have collected my data from funda.nl, which is a website that provides a marketplace for various housing options in Netherlands. For my dataset I have collected information only about apartments in Amsterdam that were being on sale on the moment of June 2026. The fact that this data is collected from a marketplace which is not controlled externally, and the people can name their own prices for their products, suggests that the value of the house is accurate as it is made up by free market forces. On the other hand, the prices are the advertsided prices rather than the actual transaction prices. This could

potentially create a bias that pushes predicted prices higher than the actual price of the worth.

The data entries for each apartment consists of more information than just living area, construction year and distance from Amsterdam, a part of it will be included in the appendix.

Furthermore, I have analyzed the data for outliers and there was none, where a statistical outlier is an entry in the dataset that value is lower than $Q_1 - \text{IQR} \times 1.5$ or higher than $Q_3 + \text{IQR} \times 1.5$. This means that the disadvantage of MSE loss, which is prioritizing outliers largely due to the squared error, might not have a significant impact on the outcome. This is supported by the results as MSE scored the lowest on the mean loss property. On the other hand, the Huber constant that achieved lowest normalized MAE is very low: 0.1. This suggests that MAE loss works efficiently for the most part except for the area around the kink, below $\delta = 0.1$.

Another problem with the data is that there might be a selection bias, as the training and testing sets were choosed randomly. The inaccuracy created by this bias might create an error of ± 20 k€ depending on which dataset model was trained and tested. To face this problem I have trained multiple models with the same parameters, with the only difference of training and testing sets and then taking the average of the results. This helped comparing loss methods because otherwise selection error might be higher than or close to the difference in results for different loss methods: ± 20 k€ $>$ ± 25 k€.

One of the limitations of this experiment is the number of inputs, features about the apartment. For this paper I have used only three features, which in my opinion has achieved incredibly high results for the amount of information provided. However, the number of features is fixed for this paper, therefore loss method is one of the other improvements I can make to this model to improve

its performance without giving it more information.

4.3 Loss Function Choice

Huber loss achieved the best results due to its compromise between MSE and MAE losses. MSE can be disproportionately impacted by extreme prediction errors. On the other hand, MAE is more robust as it treats any error the same.

4.4 Normalization

I use min-max normalization for my training. This means that all of the values get compressed into 0-1 interval, preventing large numerical values from dominating the gradients. However, this interval limits the impact of extreme values.

4.5 Alternate Approaches

Calculating the exact weights and bias could be done using Ordinary Least Squares (OLS), which is an analytical solution for this problem of finding parameters that fit a multidimensional linear function. The reason I have used gradient descent is mostly educational, as it allows for an introduction into more complex topic of machine learning with non-linear functions, which do not have a simple analytical solution.

4.6 Real-World Usefulness

To an extent, this model can be used to check prices of housing in Amsterdam. However, the inputs are extremely simplified and, although the mathematical model behind calculating the parameters might not be the most efficient, a more important limitation are the features of the apartments that were not included or simply can not be quantified, such as architectural quality, neighbourhood

reputation or buyer's preferences. The model tries to predict the behaviour of real prices, however it should not be count as a reliable source of information about worth of an apartment.

5 Conclusion

I have successfully learned the theory behind linear-regression models and trained models that use gradient descent as the the training method, to predict prices of apartments in Amsterdam on the moment of June of 2026, with the input data of living area, construction year and the distance from the center of Amsterdam. I have used three different methods of finding loss: MSE, MAE and Huber. The results I got is that Huber has performed the best according to normalized mean absolute error across 100 different dataset permutations. One the other hand, MAE performed the poorest with the worst loss and MAE_{norm} . The theory states that MSE is significantly affected by outliers, and will perform well on datasets where outliers do not have a great impact on the outcome. MAE treats every error the same with the gradient being only proportionate to the sign of the error. The results of the experiment is a great example of how the Huber loss with a correctly adjusted constant δ can combine MSE and MAE losses, and performing the best.